

Programming echip for plating

pd 20261010

echip is programmed by a series of serial writes to the chip. These are executed as USB commands to the MCU (which then takes care of the actual serial write). A single command can be executed with `echip_write.exe` (github:echip_plating/batch_scripts). A list of commands in a text file can be executed with `register_write.exe` (github:echip_plating/batch_scripts). <There seems to be overhead, so I've found it to be faster to use `register_write` if more than a few commands have to be written>

A command looks like `0x1Bmdd`, where `m` is a register address, and `dd` is the data written to that register.

m	Register or Function
0	Data register
1	Row address
2	Colum address
3	Command

If the chip is RESET, then all registers go to default values, and (generally) all have to be re-written. A template file (to use with `register_write`) is `template.txt` (github:echip_plating/doc). It consists of 3 sections, the first one sets up the general echip registers:

<code>reg_value 0x1A340 -- Reset</code>	Resets the chip
<code>reg_value 0x20001 -- Set Token</code>	(Used for token passing)
<code>reg_value 0x1B006 -- Control</code>	Writes 0x6 to the data register
<code>reg_value 0x1B278</code>	Sets the address register to 0x78 (which is the control register)
<code>reg_value 0x1B308</code>	0x308 is the "write" command, which writes what is in the data register to the address(ed) registers
<code>reg_value 0x1B00A -- BiasK1</code>	Writes 0xA to the data register
<code>reg_value 0x1B240</code>	Sets the address register to 0x40 (which is the BiasK1 register)
<code>reg_value 0x1B308</code>	0x308 is the "write" command, which writes what is in the data register to the address(ed) registers
...	And so on...

The next section (blank in the template) is for programming each of the 512 pixels.

Simplistically, there would be 512x4 lines that look like

reg_value 0x1B013 -- PixReg	Writes 0x13 to the data register (which will get sent to the selected Pixel control register)
reg_value 0x1B100 -- row	Set the pixel row to be addressed to be 0
reg_value 0x1B200 -- col	Set the pixel column to be addressed to be 0
reg_value 0x1B308 -- write	0x308 is the “write” command, which writes what is in the data register to the pixel register in the addressed row and column

The registers are static, so only registers that changed have to be updated. For example, to write the same value to **every** pixel:

reg_value 0x1B013 -- PixReg	Only need to write this once
reg_value 0x1B100 -- row	Start with row 0
reg_value 0x1B200 -- col	And column 0
reg_value 0x1B308 -- write	
reg_value 0x1B201 -- col	Then column 1
reg_value 0x1B308 -- write	
reg_value 0x1B202 -- col	Then column 2
reg_value 0x1B308 -- write	
reg_value 0x1B203 -- col	Then column 3
reg_value 0x1B308 -- write	
...	And so on for the rest of the columns
reg_value 0x1B21F -- col	Last column (31)
reg_value 0x1B308 -- write	
reg_value 0x1B101 -- row	Next row (1)
reg_value 0x1B200 -- col	And column 0
reg_value 0x1B308 -- write	
reg_value 0x1B201 -- col	Then column 1
reg_value 0x1B308 -- write	
...	And so on until done

Lastly, there are some important lines at the end

reg_value 0x1A304 -- Sync	SYNC “lines up” all the clocks
reg_value 0x1A302 -- GO	GO puts the chip into data taking mode (and turns on the blinking light of happiness . RESET takes the chip out of data taking mode)

Modifying PlateAll.py to do “clever” plating

Looking at the program

```
WriteCMD("register_write G20UP.txt")
```

This programs the chip, and sets every pixel to have a readout gain of 20 and to plate in the UP (current is sourced) direction. The plating DAC is programmed to 0.

```
WriteDAC(2) # nom. 2 for 10 nA
```

This command sets the plating DAC to “2” (roughly 5 nA per step)

```
time.sleep(45)
```

Nominal plating time is 45 seconds

```
echip_write(int("0x1A340",16)) # RESET
```

This RESETs the chip, which abruptly stops plating and turns off the plating switches

```
WriteCMD("register_write G20.txt")
```

Having done a RESET, this puts the chip “back to normal” – just for good housekeeping

“Nominal” plating is 10 nA (DAC=2) for 45 seconds.

- If things plate too fast, try DAC=1
- If things plate too slowly, try increasing time (but don’t increase DAC above 2)

Note: When the Plating DAC is 0 but the Plating Switch is ON in the pixel, there can still be current flowing. That’s why the approach here is to issue a RESET, since that turns all switches off. A “clever” plating routine might have a small number of different patterns, and just execute the steps serially:

```
WriteCMD("register_write G20groupA.txt")
```

```
WriteDAC(2) #
```

```
time.sleep(45)
```

```
echip_write(int("0x1A340",16))
```

```
WriteCMD("register_write G20groupB.txt")
```

```
WriteDAC(2) #
```

```
time.sleep(45)
```

```
echip_write(int("0x1A340",16))
```

```
WriteCMD("register_write G20.txt")
```